Princess Nourah bint Abdulrahman University
College of Computer and Information Sciences
SW 413 – Data Exploration and Visualization

ECO-GUARD

Intelligent Economic Inspector

An AI-Powered Multi-Agent System for Household Survey Data Verification

Course Code	SW 413
Group Number	Groupe 02
Team Leader	Haneen Alshuwayqi
Semester	2nd Semester –2026
Instructor	Dr. Motasem Alsawadi

Team Members

Name	Role
Haneen Alshuwayqi	Team Leader & Backend Developer
Sumaiya Balatif	Frontend Developer
Taif Alshehri	AI & Agents Engineer
Munirah Almousa	Data Analyst
Bushra Almughirah	Documentation & Visualization

Executive Summary

Eco-Guard is an AI-powered multi-agent system for verifying household survey data collected by GASTAT (General Authority for Statistics) in Saudi Arabia. Three independent agents — Demographic, Financial, and Manager — powered by Claude AI (Anthropic) analyze each record and produce a trust score, enabling analysts to flag inconsistent or fraudulent entries. The system is deployed at eco-guard.streamlit.app with a decoupled FastAPI backend on Railway, supporting manual entry, batch CSV processing, Arabic RTL dashboard, and Excel export.

1. Project Title

Eco-Guard — Intelligent Economic Inspector: An AI-Powered Multi-Agent System for Household Survey Data Verification

2. Team Members and Roles

Name	Role & Responsibilities
Haneen Alshuwayqi	Team Leader & Backend Developer – Project architecture, FastAPI deployment on Railway, GitHub management.
Sumaiya Balatif	Frontend Developer – Streamlit UI, RTL Arabic layout, CSS styling, dashboard components.
Taif Alshehri	AI & Agents Engineer – Multi-agent design, Claude API prompts, JSON parsing logic.
Munirah Almousa	Data Analyst – GASTAT benchmarks research, reference_data.json, financial thresholds.
Bushra Almughirah	Documentation & Visualization – Report, presentation, Plotly charts, Excel export.

3. Project Idea and Problem Statement

GASTAT conducts large-scale household surveys used in national planning. To elevate data quality and enhance operational efficiency, the system runs three AI agents on each record: the Demographic Agent checks age/education/occupation consistency, the Financial Agent compares income and expenses against regional benchmarks, and the Manager Agent synthesizes a final trust score with actionable recommendations.

4. Objectives

- Automate detection of inconsistencies in household survey data using AI.
- Implement a multi-agent architecture with independent specialized agents.
- Provide quantitative trust scores and recommendations for each record.
- Build an RTL Arabic-language web dashboard for non-technical users.
- Support batch CSV processing for scalable quality control workflows.
- Ensure privacy through data preprocessing before AI analysis.

5. Dataset and Data Sources

The system processes household survey records with 10 fields per record: Age, Marital Status, Education Level, Occupation, Family Size, Administrative Region, Monthly Income (SAR), Monthly Expenses (SAR), Housing Type, and Monthly Electricity Bill (SAR). Financial benchmarks are sourced from GASTAT published regional income/expenditure reports, stored in reference_data.json.

6. Workflow and Project Stages

1) Data Input: Manual entry or CSV batch upload. 2) Data Preprocessing: records cleaned and prepared before AI processing 3) Parallel Agent Analysis: Demographic and Financial agents run concurrently via ThreadPoolExecutor; Manager agent synthesizes results. 4) Visualization: Trust score gauge, bar charts, agent detail cards displayed on Arabic RTL dashboard. 5) Export & Audit: Color-coded Excel export; all operations logged in admin audit trail.

7. Prototype Explanation

Architecture

Streamlit Cloud (frontend) communicates with Railway FastAPI (backend) via HTTP REST calls through api_client.py. Each agent has an independent endpoint: POST /agent/demographic, POST /agent/financial, POST /agent/manager. Each endpoint preprocesses the record. builds a specialized prompt, calls Claude Sonnet API, and returns structured JSON.

Authentication

Role-based auth with PBKDF2 password hashing. Two roles: Field Researcher and System Administrator. Admin has user management and audit log access.

8. Screenshots

The following screenshots illustrate the key screens of Eco-Guard:



Figure1 : Login Screen — Authentication page with Eco-Guard branding

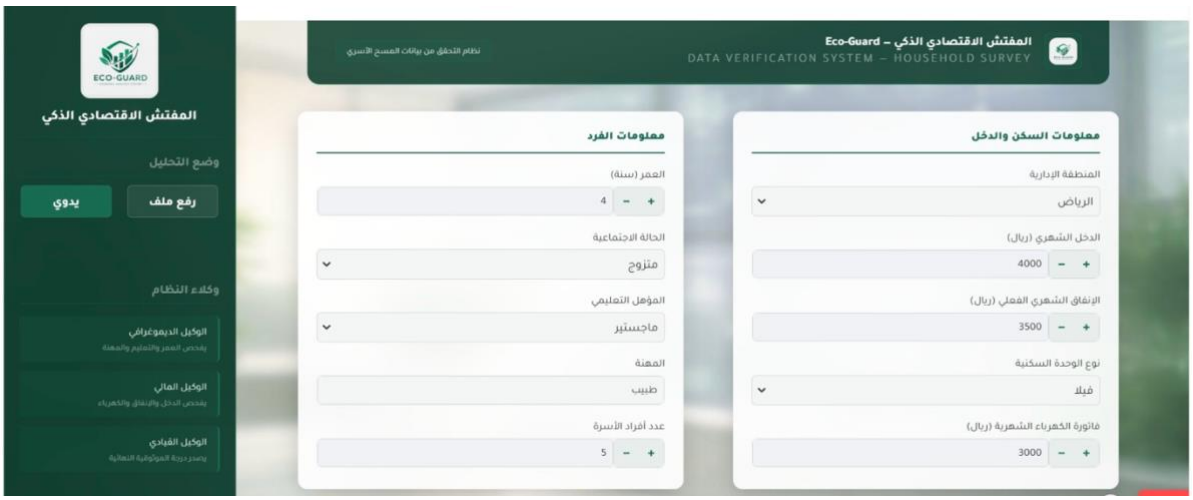


Figure2 : Manual Input Form — Two-column data entry form

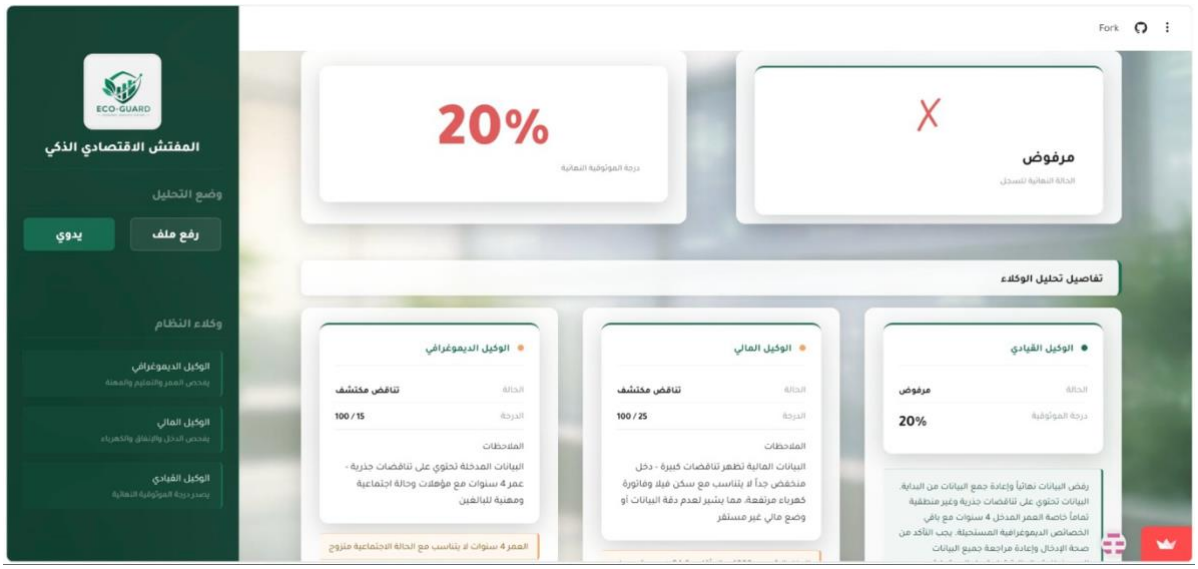


Figure3 : Analysis Results — Trust score, agent cards, and recommendations



Figure4 : Analysis Results — Trust score, agent cards, and recommendations

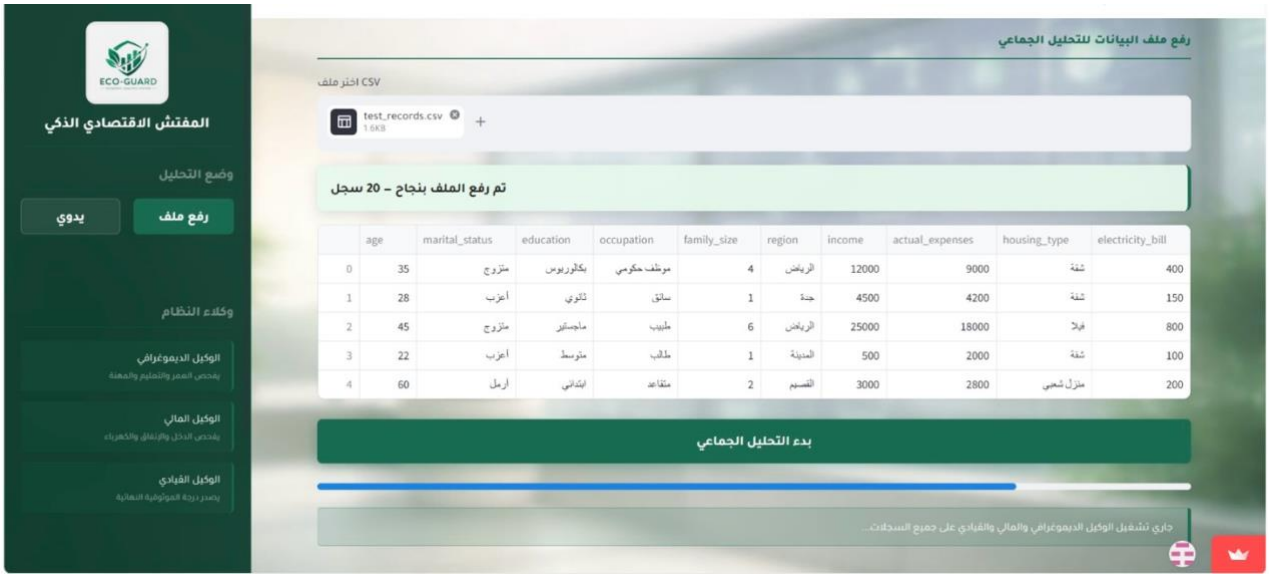


Figure5 : Batch Analysis — CSV upload results

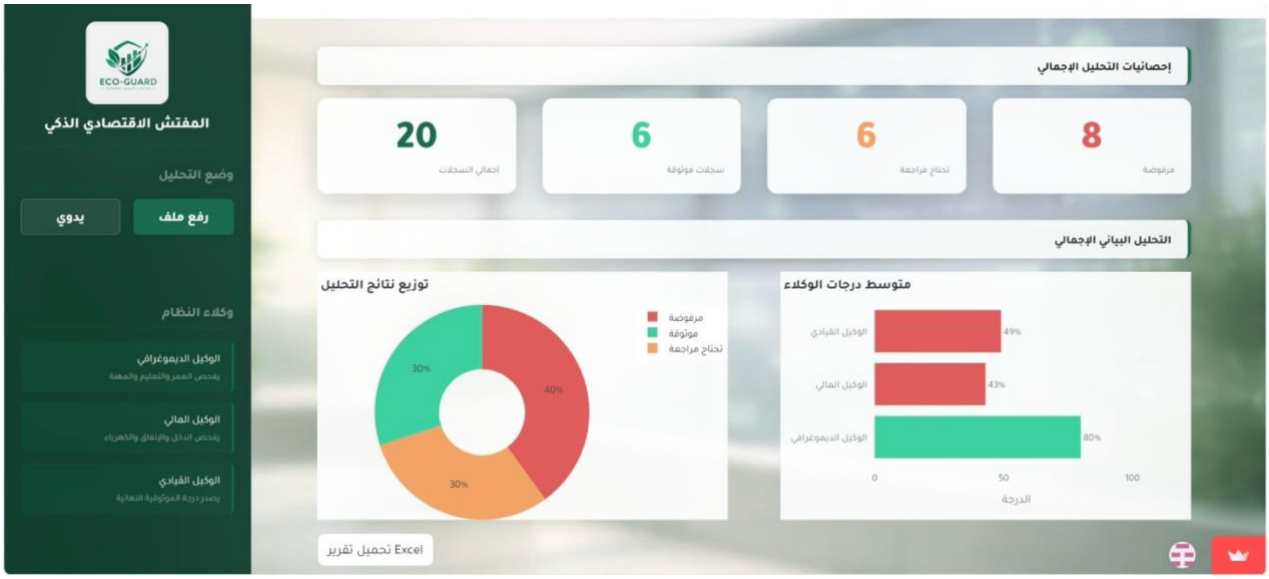


Figure6 : Batch Analysis — CSV upload results with statistics and charts

9. Project Links

Resource	Link
Live Application	https://eco-guard.streamlit.app
GitHub Repository	https://github.com/Haneen-Alshuwayqi/Eco-Guard

Table1 : Eco-Guard Deployment Links

10. Results, Observations, and Improvements

In testing with 20 sample records, agents correctly flagged: age-education inconsistencies (3-year-old with doctorate), income 99% below GASTAT regional median, expense-to-income ratio exceeding 90%, and electricity bills inconsistent with housing type. Key improvements: migrated to decoupled architecture (Streamlit + Railway FastAPI), upgraded AI from Groq/LLaMA to Claude

Sonnet, added parallel agent execution reducing analysis time by ~60%, and implemented PBKDF2 authentication.

11. Challenges and Solutions

Challenge	Solution
Railway 502 errors	Fixed UTF-8 encoding error in Arabic text; moved client init inside call_llm()
API key not loading on Railway	Moved Anthropic client initialization inside the function to read env vars at runtime
PORT misconfiguration	Added railway.toml with explicit $PORT variable
Arabic RTL layout	Applied CSS direction:rtl with Tajawal Google Font across all components
JSON parsing failures	Built multi-attempt parser with regex fallback for all critical fields

Table2 : Challenges and Solutions

12. Conclusion

Eco-Guard demonstrates AI and multi-agent systems applied to a real GASTAT data quality challenge. The decoupled architecture (Streamlit frontend + Railway FastAPI backend) with three specialized Claude AI agents delivers a scalable, privacy-preserving verification solution. The system is fully operational at eco-guard.streamlit.app. Future work includes integration with GASTAT live systems and additional regional agents.